Assignment No : 01

Execute the commands for the following case scenarios in your Linux terminal:

1. Create two empty directories named dir1, and dir2 on Desktop.

```
labex:project/ $ cd ~/Desktop
labex:Desktop/ $ mkdir dir1 dir2
```

2. Display present working directory on the terminal.

```
labex:Desktop/ $ pwd
/home/labex/Desktop
```

3. Navigate to dir1 and create an empty file named file1.txt.

```
labex:Desktop/ $ cd dir1
labex:dir1/ $ touch file1.txt
```

4. Add “Hello, {Your Name}” to file1.txt using Linux text editor.

```
labex:dir1/ $ echo "Hello, Vishwa" > file1.txt
```

5. Display the contents of file1.txt on the terminal.

```
labex:dir1/ $ cat file1.txt
Hello, Vishwa
```

6. Create a copy of file1.txt named file2.txt and move it to directory dir2.

```
labex:dir1/ $ cp file1.txt file2.txt
labex:dir1/ $ mv file2.txt ../dir2
```

7. Rename file.txt to oldfile.txt.

```
labex:dir1/ $ mv file1.txt oldfile.txt
```

8. Move dir1 to dir2.

```
labex:dir1/ $ cd ..
labex:Desktop/ $ mv dir1 dir2
```

9. Delete dir2.

```
labex:Desktop/ $ rm -r dir2
```

10. Display “Yeah!! I have completed first Lab Assignment of Operating System course” on terminal.

```
labex:Desktop/ $ echo "Yeah! I have completed first Lab Assignment of Operating System course"
Yeah! I have completed first Lab Assignment of Operating System course
```

```
labex:project/ $ cd ~/Desktop
labex:Desktop/ $ mkdir dir1 dir2
labex:Desktop/ $ pwd
/home/labex/Desktop
labex:Desktop/ $ cd dir1
labex:dir1/ $ touch file1.txt
labex:dir1/ $ echo "Hello, Vishwa" > file1.txt
labex:dir1/ $ cat file1.txt
Hello, Vishwa
labex:dir1/ $ cp file1.txt file2.txt
labex:dir1/ $ mv file2.txt ../dir2
labex:dir1/ $ mv file1.txt oldfile.txt
labex:dir1/ $ cd ..
labex:Desktop/ $ mv dir1 dir2
labex:Desktop/ $ rm -r dir2
labex:Desktop/ $ echo "Yeah, I have completed first Lab Assignment of Operating System course"
Yeah, I have completed first Lab Assignment of Operating System course
```

Assignment No : 02

Linux Practical – Advanced Linux Commands

Aim: To understand and use basic advanced Linux commands for file management, permissions, compression, networking, and process control.

Practical Question:

Perform the following tasks using appropriate Linux commands:

1. **Create a directory named linux_practical on the Desktop.**

```
labex:project/ $ mkdir ~/Desktop/linux_practical
labex:project/ $ cd  ~/Desktop/linux_practical
```

2. **Inside the directory, create a text file named data.txt and add some sample text to it.**

```
labex:linux_practical/ $ echo "Linux commands are powerful." > data.txt
labex:linux_practical/ $ echo "Grep is used for searching text." >> data.txt
labex:linux_practical/ $ echo "Chmod changes file permissions." >> data.txt
```

3. **Use the grep command to search for a specific word from data.txt and display the matching lines.**

```
labex:linux_practical/ $ grep "Grep" data.txt
Grep is used for searching text.
```

4. **Change the permission of data.txt so that the owner has read and write permission using the chmod command.**

```
labex:linux_practical/ $ chmod u+rw data.txt
```

5. **Change the ownership of data.txt to another user using the chown command.** The `chown` command in Linux, short for "change owner," is a fundamental utility used by system administrators and users with sufficient privileges (usually root/sudo) to manage file system access control. Its primary use is to transfer ownership of a file or directory to a new user and/or group.

```
labex:linux_practical/ $ sudo chown newuser data.txt
chown: invalid user: 'newuser'
```

6. Compress the file data.txt using the gzip command.

```
labex:linux_practical/ $ gzip data.txt
```

7. Create an archive of the directory linux_practical using the tar command.

```
labex:Desktop/ $ tar -czvf linux_practical.tar.gz linux_practical/
linux_practical/
linux_practical/data.txt.gz
labex:Desktop/ $ cd linux_practical/
```

8. Download a sample file from the internet using the wget command.

```
labex:linux_practical/ $ wget https://example.com/index.html
--2025-12-23 15:02:40--  https://example.com/index.html
Resolving example.com (example.com)... 104.18.26.120, 104.18.27.120, 2606:4700::6812:1b78, ...
Connecting to example.com (example.com)|104.18.26.120|:443...
```

9. Install any package (for example: tree) using apt or apt-get command.

```
labex:linux_practical/ $ sudo apt update
Get:1 http://mirrors.cloud.aliyuncs.com/ubuntu jammy InRelease [270 kB]
Get:2 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-backports InRelease [127 kB]
Get:4 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://mirrors.cloud.aliyuncs.com/ubuntu jammy/main amd64 Packages [1,792 kB]
Get:6 http://mirrors.cloud.aliyuncs.com/ubuntu jammy/restricted amd64 Packages [164 kB]
Get:7 http://mirrors.cloud.aliyuncs.com/ubuntu jammy/multiverse amd64 Packages [266 kB]
Get:8 http://mirrors.cloud.aliyuncs.com/ubuntu jammy/universe amd64 Packages [17.5 MB]
Get:9 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-updates/main amd64 Packages [3,966 kB]
Get:10 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-updates/multiverse amd64 Packages [69.2 kB]
Get:11 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-updates/restricted amd64 Packages [6,411 kB]
Get:12 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-updates/universe amd64 Packages [1,598 kB]
Get:13 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-backports/main amd64 Packages [83.9 kB]
Get:14 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-backports/universe amd64 Packages [37.2 kB]
Get:15 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-security/main amd64 Packages [3,633 kB]
Get:16 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-security/restricted amd64 Packages [6,205 kB]
Get:17 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-security/universe amd64 Packages [1,287 kB]
Get:18 http://mirrors.cloud.aliyuncs.com/ubuntu jammy-security/multiverse amd64 Packages [60.9 kB]
Fetched 43.7 MB in 3s (13.5 MB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
313 packages can be upgraded. Run 'apt list --upgradable' to see them.
labex:linux_practical/ $ sudo apt install tree
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tree is already the newest version (2.0.2-1).
The following packages were automatically installed and are no longer required:
  gir1.2-libxfce4util-1.0 gir1.2-xfconf-0 gsfonts gsfonts-x11 libdbus-glib-1-2
  libgdk-pixbuf-xlib-2.0-0 libjpeg-turbo-progs miscfiles xscreensaver-data
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 313 not upgraded.
```

10. Display network-related information using the netstat command.

```
labex:linux_practical/ $ netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.11:34671        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:3001            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:3002            0.0.0.0:*               LISTEN
tcp6       0      0 :::22                   :::*                    LISTEN
udp        0      0 127.0.0.11:32973        0.0.0.0:*
udp        0      0 0.0.0.0:3001            0.0.0.0:*
```

11. Find the IP address of a website using the nslookup command.

```
labex:linux_practical/ $ nslookup google.com
Server:         127.0.0.11
Address:        127.0.0.11#53

Non-authoritative answer:
Name:   google.com
Address: 142.251.46.238
Name:   google.com
Address: 2607:f8b0:4005:813::200e
```

12. Display all running processes and terminate any one process using the kill command.

```
labex:linux_practical/ $ ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0  11204  3792 ?        Ss   14:33   0:00 /bin/bash /etc/shiyanlou/sbin/init.sh
root          21  0.0  0.1  40824 25956 ?        S    14:33   0:00 /usr/bin/python3 /usr/bin/supervisord -n
root          22  0.0  0.0  15424  7044 ?        S    14:33   0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
root          23  0.0  0.0  14040  3964 ?        S    14:33   0:00 su labex -c vncserver -fg -disableBasicAuth -publicIp 0.0.0.0
labex         24  0.0  0.1  40308 30740 ?        Ss   14:33   0:00 /usr/bin/perl /usr/bin/vncserver -fg -disableBasicAuth -publicIp 0.0.0.0
labex         36  2.5  0.8 878692 138272 ?       Sl   14:33   1:13 /usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.0.0.0 -WebpVideoQuality -1 -auth /home/labex
/.Xauthority -websocketPort 3001 -BlacklistTimeout 10 -AcceptCutText 1 -drinode /dev/dri/renderD128 -QueryConnectTimeout 10 -JpegVideoQuality -1 -FrameRate 60
 -DLP_RegionAllowClick 0 -DLP_RegionAllowRelease 0 -SendCutText 1 -ImprovedHextile 1 -fp /usr/share/fonts/X11//misc,/usr/share/fonts/X11//type1 -IdleTimeout 0
 -MaxVideoResolution 1920x1080 -DisconnectClients 0 -VideoScaling 2 -DLP_ClipAcceptMax 0 -DLP_Log off -PrintVideoArea 0 -DLP_KeyRateLimit 0 -DynamicQualityMax
 8 -UseIPv4 1 -AcceptPointerEvents 1 -DLP_ClipTypes chromium/x-web-custom-data,text/html,image/png -BlacklistThreshold 5 -DynamicQualityMin 7 -VideoTime 5 -DL
P_ClipDelay 0 -VideoOutTime 3 -cert /etc/ssl/certs/ssl-cert-snakeoil.pem -sslOnly 0 -AvoidShiftNumLock 0 -MaxDisconnectionTime 0 -TreatLossless 10 -RawKeyboar
d 0 -VideoArea 45 -interface 0.0.0.0 -MaxConnectionTime 0 -UseIPv6 0 -udpFullFrameFrequency 0 -key /etc/ssl/private/ssl-cert-snakeoil.key -MaxIdleTime 0 -Log
*:stdout:100 -CompareFB 2 -RectThreads 0 -httpd /usr/share/kasmvnc/custom_www -http-header Cross-Origin-Embedder-Policy=require-corp -http-header Cross-Origin
-Opener-Policy=same-origin -depth 24 -QueryConnect 0 -AcceptSetDesktopSize 1 -SendPrimary 0 -KasmPasswordFile /home/labex/.kasmpasswd -IgnoreClientSettingsKas
m 1 -desktop 694a37bfa8b45b9cfb75fc8e:1 (labex) -AllowOverride AcceptPointerEvents,SendCutText,AcceptCutText,SendPrimary -DLP_ClipSendMax 0 -geometry 1024x768
 -AcceptKeyEvents 1 -rfbwait 30000 -rfbauth /home/labex/.vnc/passwd -rfbport 5901
labex         44  0.0  0.0  11204  3456 ?        S    14:33   0:00 sh -c { echo 'Running /home/labex/.vnc/xstartup'; /home/labex/.vnc/xstartup; } >> '/home/la
bex/.vnc/694a37bfa8b45b9cfb75fc8e:1.log' 2>&1
labex         45  0.0  0.0  11204  1936 ?        S    14:33   0:00 sh -c { echo 'Running /home/labex/.vnc/xstartup'; /home/labex/.vnc/xstartup; } >> '/home/la
bex/.vnc/694a37bfa8b45b9cfb75fc8e:1.log' 2>&1
labex         46  0.0  0.4 454064 75568 ?        Sl   14:33   0:00 xfce4-session
```

```
labex:linux_practical/ $ kill 24
```

13. Execute one command using sudo to demonstrate administrative privileges.

```
labex:linux_practical/ $ sudo whoami
root
```

14. Display the following message on the terminal:
"Linux Advanced Commands Practical Completed Successfully!"

```
labex:linux_practical/ $ echo "Linux Advanced Commands Practical Completed Successfully"
Linux Advanced Commands Practical Completed Successfully
```

Assignment No : 03

Write shell scripts for the following:

1. Read two numbers and perform Arithmetic operations (Addition, Subtraction, Division, Multiplication) on them.

Input:

```
labex:Desktop/ $ #!/bin/zsh

echo "Enter first number:"
read a
echo "Enter second number:"
read b

echo "Addition = $((a + b))"
echo "Subtraction = $((a - b))"
echo "Multiplication = $((a * b))"

if [ $b -ne 0 ]; then
    echo "Division = $((a / b))"
else
    echo "Division not possible"
fi
```

Output:

```
Enter first number:
25
Enter second number:
12
Addition = 37
Subtraction = 13
Multiplication = 300
Division = 2
labex:Desktop/ $
```

2. Read a number and find factorial of that number.

Input:

```
labex:Desktop/ $ #!/bin/bash

echo "Enter a number:"
read n

fact=1
for (( i=1; i<=n; i++ ))
do
    fact=$((fact * i))
done

echo "Factorial of $n is $fact"
```

Output:

```
Enter a number:
5
Factorial of 5 is 120
labex:Desktop/ $
```

3. Read a number N and find Nth Fibonacci number.

Input:

```
labex:Desktop/ $ #!/bin/bash

echo "Enter value of N:"
read n

a=0
b=1

if [ $n -eq 0 ]; then
    echo "Fibonacci number is 0"
elif [ $n -eq 1 ]; then
    echo "Fibonacci number is 1"
else
    for (( i=2; i<=n; i++ ))
    do
        c=$((a + b))
        a=$b
        b=$c
    done
    echo "Nth Fibonacci number is $b"
fi
```

Output:

```
Enter value of N:
7
Nth Fibonacci number is 13
labex:Desktop/ $
```

4. Read a string S and find if it is palindrome Input:

```
labex:Desktop/ $ #!/bin/bash

echo "Enter a string:"
read s

rev=$(echo "$s" | rev)

if [ "$s" = "$rev" ]; then
    echo "Palindrome"
else
    echo "Not Palindrome"
fi
```

Output:

```
Enter a string:
madam
Palindrome
labex:Desktop/ $
```

5. Input a string as password and validate that password:

a. It should be Minimum of 8 characters.

b. It should have at least one digit.

c. It should have at least one uppercase letter.

Input:

```
labex:Desktop/ $ #!/bin/bash

echo "Enter password:"
read password

len=${#password}

if [ $len -lt 8 ]; then
    echo "Password must be at least 8 characters"
elif [[ ! $password =~ [0-9] ]]; then
    echo "Password must contain at least one digit"
elif [[ ! $password =~ [A-Z] ]]; then
    echo "Password must contain at least one uppercase letter"
else
    echo "Password is valid"
fi
```

Output:

```
Enter password:
Pass8917
Password is valid
labex:Desktop/ $
```

Assignment No : 04

As a part of this assignment, you need to simulate any two CPU Scheduling algorithm from the following:

1. First come first serve
2. Round Robin
3. Shortest Job First

In particular, you must complete the following tasks:

1. Draw the Gantt Chart for finding the competition time for each process.
2. Find turnaround time, waiting time and response time for each process.
3. Display processID, arrivalTime, BurstTime, competition time, turnaround time, waiting time and response time in tabular format.
4. Find average waiting time and average turnaround time and average response time.

You need to find out the above details for the following test cases:

ProcessID	ArrivalTime (ms)	BurstTime(ms)
P1	2	3
P2	0	5
P3	1	2
P4	0	4
	1	

For round robin algorithm, consider time quantum as 2ms.

Please feel free to test on your own test cases also.

Code:-

```
import java.util.*;

class Process {
String pid;
   int at, bt, ct, tat, wt, rt;
int remaining; boolean
started = false;
Process(String pid, int at,
int bt) {
      this.pid = pid;
this.at = at;       this.bt
= bt;
      this.remaining = bt;
   }
}

public class CPUScheduling {

   static void printTable(List<Process> pList) {
double awt = 0, atat = 0, art = 0;

      System.out.println("PID\tAT\tBT\tCT\tTAT\tWT\tRT");
      for (Process p : pList) {
p.tat = p.ct - p.at;
         p.wt = p.tat - p.bt;

         awt += p.wt;
atat += p.tat;
         art += p.rt;

         System.out.println(p.pid + "\t" + p.at + "\t" + p.bt + "\t" +
p.ct + "\t" + p.tat + "\t" + p.wt + "\t" + p.rt);
      }

      int n = pList.size();
      System.out.println("Average WT = " + awt / n);
      System.out.println("Average TAT = " + atat / n);
      System.out.println("Average RT = " + art / n);
   }
```

```
   static void fcfs(List<Process> list) {
System.out.println("\n--- FCFS ---");

      list.sort(Comparator.comparingInt(p -> p.at));
int time = 0;

      System.out.print("Gantt Chart: ");
for (Process p : list) {            if (time <
p.at) time = p.at;            p.rt = time -
p.at;
         System.out.print("| " + p.pid + " ");
time += p.bt;
         p.ct = time;

   }
      System.out.println("|");

      printTable(list);
   }

static void sjf(List<Process> list) {
      System.out.println("\n--- SJF (Non-Preemptive) ---");

      int time = 0, completed = 0, n = list.size();
      boolean[] done = new boolean[n];

      System.out.print("Gantt Chart: ");
while (completed < n) {
         int idx = -1, minBT = Integer.MAX_VALUE;

         for (int i = 0; i < n; i++) {
Process p = list.get(i);
            if (!done[i] && p.at <= time && p.bt < minBT) {
               minBT = p.bt;
idx = i;
            }
         }

         if (idx == -1) {
time++;               continue;
         }
```

```
            Process p = list.get(idx);
p.rt = time - p.at;
            System.out.print("| " + p.pid + " ");
time += p.bt;            p.ct = time;

            done[idx] = true;
            completed++;
        }
        System.out.println("|");

        printTable(list);
    }


    static void roundRobin(List<Process> list, int tq) {
System.out.println("\n--- Round Robin (TQ = " + tq + ") ---");

        Queue<Process> q = new LinkedList<>();
        int time = 0, completed = 0, n = list.size();

        list.sort(Comparator.comparingInt(p -> p.at));
int i = 0;

        System.out.print("Gantt Chart: ");
        while (completed < n) {

            while (i < n && list.get(i).at <= time) {
q.add(list.get(i));
                i++;
            }

            if (q.isEmpty()) {
time++;
                continue;
            }

            Process p = q.poll();

            if (!p.started) {
                p.rt = time - p.at;
                p.started = true;
            }
```

```
        System.out.print("| " + p.pid + " ");

        int exec = Math.min(tq, p.remaining);
p.remaining -= exec;          time += exec;

        while (i < n && list.get(i).at <= time) {
q.add(list.get(i));
            i++;
        }

        if (p.remaining > 0)
q.add(p);          else {
            p.ct = time;
            completed++;
        }
    }
    System.out.println("|");

    printTable(list);
  }


  public static void main(String[] args) {

    List<Process> base = Arrays.asList(
new Process("P1", 2, 3),              new
Process("P2", 0, 5),              new
Process("P3", 1, 2),              new
Process("P4", 0, 4),              new
Process("P5", 1, 1)
    );

    fcfs(cloneList(base));
sjf(cloneList(base));
    roundRobin(cloneList(base), 2);
  }

  static List<Process> cloneList(List<Process> list) {
List<Process> copy = new ArrayList<>();          for
(Process p : list)
        copy.add(new Process(p.pid, p.at, p.bt));
```

```
        return copy;
    }
}
```

OUTPUT:-

```
--- FCFS ---
Gantt Chart: | P2 | P4 | P3 | P5 | P1 |
PID     AT      BT      CT      TAT     WT      RT
P2      0       5       5       5       0       0
P4      0       4       9       9       5       5
P3      1       2       11      10      8       8
P5      1       1       12      11      10      10
P1      2       3       15      13      10      10
Average WT = 6.6
Average TAT = 9.6
Average RT = 6.6

--- SJF (Non-Preemptive) ---
Gantt Chart: | P4 | P5 | P3 | P1 | P2 |
PID     AT      BT      CT      TAT     WT      RT
P1      2       3       10      8       5       5
P2      0       5       15      15      10      10
P3      1       2       7       6       4       4
P4      0       4       4       4       0       0
P5      1       1       5       4       3       3
Average WT = 4.4
Average TAT = 7.4
Average RT = 4.4

--- Round Robin (TQ = 2) ---
Gantt Chart: | P2 | P4 | P3 | P5 | P1 | P2 | P4 | P1 | P2 |
PID     AT      BT      CT      TAT     WT      RT
P2      0       5       15      15      10      0
P4      0       4       13      13      9       2
P3      1       2       6       5       3       3
P5      1       1       7       6       5       5
P1      2       3       14      12      9       5
Average WT = 7.2
Average TAT = 10.2
Average RT = 3.0
```

Assignment No : 05

Memory Allocation Techniques: First Fit, Best Fit, Worst Fit. import java.util.*;

```
public class Main {
   public static void firstFit(int[] blockSize, int[] processSize) { int n =
      processSize.length;

      System.out.println("\n--- First Fit ---");

      for (int i = 0; i < n; i++) {
         boolean allocated = false;

         for (int j = 0; j < blockSize.length; j++) {
            if (blockSize[j] >= processSize[i]) {
               System.out.println("Process " + i + " allocated to block " + j); blockSize[j] -=
               processSize[i];  allocated = true;  break;
            }
         }

         if (!allocated)
            System.out.println("Process " + i + " not allocated");
      }
   }

   public static void bestFit(int[] blockSize, int[] processSize) { int n =
      processSize.length;

      System.out.println("\n--- Best Fit ---");

      for (int i = 0; i < n; i++) {
         int bestIndex = -1;

         for (int j = 0; j < blockSize.length; j++) {
            if (blockSize[j] >= processSize[i]) {  if (bestIndex == -1 || blockSize[j] <
                  blockSize[bestIndex]) {  bestIndex = j;
               }
            }
         }

         if (bestIndex != -1) {
                                          System.out.println("Process " + i + " allocated to block " + bestIndex);
```

```
            blockSize[bestIndex] -= processSize[i];
        } else {
            System.out.println("Process " + i + " not allocated");
        }
    }
}

public static void worstFit(int[] blockSize, int[] processSize) { int n =
    processSize.length;

    System.out.println("\n--- Worst Fit ---");

    for (int i = 0; i < n; i++) {
        int worstIndex = -1;

        for (int j = 0; j < blockSize.length; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (worstIndex == -1 || blockSize[j] > blockSize[worstIndex]) { worstIndex = j;
            }
            }
        }

        if (worstIndex != -1) {
            System.out.println("Process " + i + " allocated to block " + worstIndex);
            blockSize[worstIndex] -= processSize[i];
        } else {
            System.out.println("Process " + i + " not allocated");
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of memory blocks: ");  int
    m = sc.nextInt();

    int[] blockSize = new int[m];
    System.out.println("Enter block sizes:");
    for (int i = 0; i < m; i++) {
        blockSize[i] = sc.nextInt();
    }

    System.out.print("\nEnter number of processes: ");  int
    n = sc.nextInt();

    int[] processSize = new int[n];
```

```
        System.out.println("Enter process sizes:");
        for (int i = 0; i < n; i++) {
            processSize[i] = sc.nextInt();
        }
        // Copy arrays (so each algorithm uses original data) int[] block1 =
        blockSize.clone();  int[] block2 = blockSize.clone();  int[] block3 =
        blockSize.clone();

        firstFit(block1, processSize);  bestFit(block2,
        processSize);
        worstFit(block3, processSize);
    }
}
```

Output :

```
Enter number of memory blocks: 5
Enter block sizes:
100 500 200 300 600

Enter number of processes: 4
Enter process sizes:
212 417 112 426

--- First Fit ---
Process 0 allocated to block 1
Process 1 allocated to block 4
Process 2 allocated to block 1
Process 3 not allocated

--- Best Fit ---
Process 0 allocated to block 3
Process 1 allocated to block 1
Process 2 allocated to block 2
Process 3 allocated to block 4

--- Worst Fit ---
Process 0 allocated to block 4
Process 1 allocated to block 1
Process 2 allocated to block 4
Process 3 not allocated
```

Assignment No : 06

Implement Banker's algorithm for deadlock avoidance. Given the Allocation matrix, Max Need matrix and Available vector, find out whether the system is in safe state or not. If yes, then provide any safe sequence. If no then print "The system is in deadlock state." import java.util.*;

```
public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter number of processes: ");  int
      n = sc.nextInt();

      System.out.print("Enter number of resources: ");  int
      m = sc.nextInt();

      int[][] alloc = new int[n][m];
      int[][] max = new int[n][m];
      int[][] need = new int[n][m];  int[]
      avail = new int[m];

      // Allocation Matrix
      System.out.println("\nEnter Allocation Matrix:");  for(int
      i = 0; i < n; i++) {
         System.out.print("Process P" + i + ": ");
         for(int j = 0; j < m; j++) {
            alloc[i][j] = sc.nextInt();
         }
      }

      // Max Matrix
      System.out.println("\nEnter Max Matrix:");  for(int
      i = 0; i < n; i++) {
         System.out.print("Process P" + i + ": ");
         for(int j = 0; j < m; j++) {
            max[i][j] = sc.nextInt();
         }
      }
```

```
// Available Resources
System.out.println("\nEnter Available Resources:");  for(int
i = 0; i < m; i++) {
   System.out.print("Resource " + i + ": ");
   avail[i] = sc.nextInt();
}
// Need = Max - Allocation
for(int i = 0; i < n; i++) {
   for(int j = 0; j < m; j++) {
      need[i][j] = max[i][j] - alloc[i][j];
   }
}

boolean[] finish = new boolean[n];
int[] safe = new int[n];  int count =
0;

while(count < n) {
   boolean found = false;

   for(int i = 0; i < n; i++) {
      if(!finish[i]) {
         int j;  for(j = 0; j < m;
         j++) {
            if(need[i][j] > avail[j]) break;
         }

         if(j == m) {
            // Add allocated resources back  for(int
            k = 0; k < m; k++) {
               avail[k] += alloc[i][k];
            }

            safe[count++] = i;
            finish[i] = true;
            found = true;
         }
      }
   }

   if(!found) {
      System.out.println("\nThe system is in deadlock state."); return;
   }
}

// Safe sequence
```

```
        System.out.print("\nSystem is in SAFE state.\nSafe sequence: "); for(int
        i = 0; i < n; i++) {
            System.out.print("P" + safe[i] + " ");
        }
    }
}
```

Output :

```
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
Process P0: 0 1 0
Process P1: 2 0 0
Process P2: 3 0 2
Process P3: 2 1 1
Process P4: 0 0 2

Enter Max Matrix:
Process P0: 7 5 3
Process P1: 3 2 2
Process P2: 9 0 2
Process P3: 2 2 2
Process P4: 4 3 3

Enter Available Resources:
Resource 0: 3
Resource 1: 3
Resource 2: 2

System is in SAFE state.
Safe sequence: P1 P3 P4 P0 P2
```

Assignment No : 07

Implement the First In First Out (FIFO), Least Recently Used (LRU) and Optimal Page Replacement algorithms. Given the number of frames in main memory and a reference string, simulate the above algorithms and find out the number of Page Faults and Page Hits.
Example: Number of frames = 4
Reference String = 5, 1, 2, 4, 3, 7, 2, 1, 0, 5, 3

```
import java.util.*;

public class PageReplacement {

   static void fifo(int[] pages, int frames) {
int[] memory = new int[frames];
      Arrays.fill(memory, -1);

      int index = 0, hits = 0, faults = 0;

      for (int page : pages) {
boolean found = false;

         for (int i = 0; i < frames; i++) {
if (memory[i] == page) {
found = true;                    hits++;
break;
            }
         }
```

```
if (!found) {
    memory[index] = page;            index
    = (index + 1) % frames;
    faults++;
            }
        }

        System.out.println("FIFO -> Hits: " + hits + ", Page Faults: " + faults);
      }

      static void lru(int[] pages, int frames) {
    int[]  memory  =  new  int[frames];    int[]
    time = new int[frames];
        Arrays.fill(memory, -1);

        int counter = 0, hits = 0, faults = 0;

        for (int page : pages) {
    boolean found = false;

            for (int i = 0; i < frames; i++) {
    if (memory[i] == page) {
    found = true;                hits++;
    time[i] = ++counter;
    break;
                }
            }

            if (!found) {
                int minTime = Integer.MAX_VALUE, pos = 0;

                for (int i = 0; i < frames; i++) {
    if (memory[i] == -1) {                    pos
    = i;                    break;
                    }
                    if (time[i] < minTime) {
    minTime = time[i];                    pos
    = i;
                    }
                }
```

```
            memory[pos] = page;
time[pos] = ++counter;
faults++;
        }
    }

    System.out.println("LRU  -> Hits: " + hits + ", Page Faults: " + faults);
  }

  static void optimal(int[] pages, int frames) {
int[] memory = new int[frames];
    Arrays.fill(memory, -1);

    int hits = 0, faults = 0;

    for (int i = 0; i < pages.length; i++) {
int page = pages[i];          boolean
found = false;

        for (int j = 0; j < frames; j++) {
if (memory[j] == page) {
found = true;                 hits++;
break;
            }
        }

        if (!found) {
int pos = -1;

                        for (int j = 0; j < frames; j++) {
if (memory[j] == -1) {                    pos = j;
break;
                }
            }

            if (pos == -1) {
int farthest = i + 1;
pos = 0;

                for (int j = 0; j < frames; j++) {
```

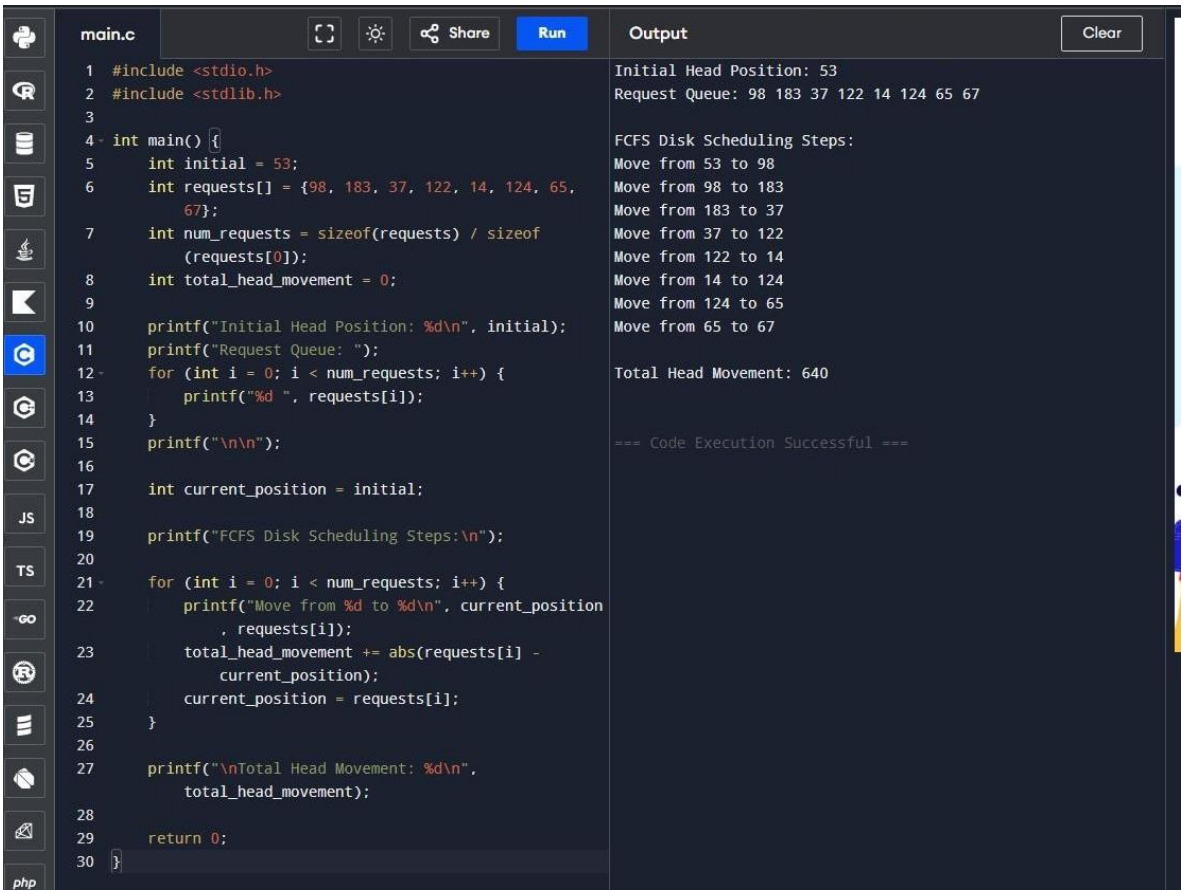
```
                    int k;
                    for (k = i + 1; k < pages.length; k++) {
if (memory[j] == pages[k]) {
                        if (k > farthest) {
farthest = k;                           pos
= j;
                        }
break;
                      }
                    }

                    if (k == pages.length) {
pos = j;                        break;
                    }
                  }
              }

              memory[pos] = page;
faults++;
          }
      }

      System.out.println("Optimal -> Hits: " + hits + ", Page Faults: " + faults);
   }

   public static void main(String[] args) {
int[] pages = {5, 1, 2, 4, 3, 7, 2, 1, 0, 5, 3};
int frames = 4;

      fifo(pages,         frames);
lru(pages,                frames);
optimal(pages, frames);
   }
}
```

OUTPUT:

```
FIFO -> Hits: 1, Page Faults: 10
LRU  -> Hits: 1, Page Faults: 10
Optimal -> Hits: 3, Page Faults: 8
```

ASSIGNMENT -8

FIFS Disk scheduling




```
#include <stdio.h>

#include <stdlib.h>


int main() {

    int initial = 53;

    int requests[] = {98, 183, 37, 122, 14, 124, 65, 67};

    int num_requests = sizeof(requests) / sizeof(requests[0]);

    int total_head_movement = 0;


    printf("Initial Head Position: %d\n", initial);

    printf("Request Queue: ");

    for (int i = 0; i < num_requests; i++) {

        printf("%d ", requests[i]);
```

```
    }
    printf("\n\n");

    int current_position = initial;

    printf("FCFS Disk Scheduling Steps:\n");

    for (int i = 0; i < num_requests; i++) {
        printf("Move from %d to %d\n", current_position, requests[i]);
        total_head_movement += abs(requests[i] - current_position);
        current_position = requests[i];
    }

    printf("\nTotal Head Movement: %d\n", total_head_movement);

    return 0;
}
```

OUTPUT

```
Initial Head Position: 53
Request Queue: 98 183 37 122 14 124 65 67

FCFS Disk Scheduling Steps:
Move from 53 to 98
Move from 98 to 183
Move from 183 to 37
Move from 37 to 122
Move from 122 to 14
Move from 14 to 124
Move from 124 to 65
```

Move from 65 to 67

Total Head Movement: 640

=== Code Execution Successful ==